



Curso Javascript básico

Aula 03 - Declaração de Variáveis



O que são expressões?

Uma expressão consiste em qualquer unidade válida de código que é resolvida como um valor.

Conceitualmente, existem dois tipos de expressões: aquelas que atribuem um valor a uma variável e aquelas que simplesmente possuem um valor.

A expressão `x = 7` é um exemplo. Esta expressão utiliza o operador `=` para atribuir o valor sete à variável `x`. A expressão em si é avaliada como sete.

O código `3 + 4` também é um exemplo. Esta expressão utiliza o operador `+` para somar três e quatro sem atribuir o resultado, sete, a uma variável.



Tipos de expressões

O JavaScript possui as seguintes categorias de expressão:

Aritmética: é avaliada como um número, por exemplo 3.14159. (Geralmente utiliza operadores aritméticos).

String: é avaliada como uma string de caracteres, por exemplo, "Fred" ou "234". (Geralmente utiliza operadores de string).

Lógica: é avaliada como verdadeira ou falsa. (Costuma envolver operadores lógicos).

Expressões primárias: Palavras reservadas e expressões gerais do JavaScript.

Expressão lado esquerdo: atribuição à esquerda de valores.



O que são variáveis?

Variáveis são espaços de memória do computador destinados a dados que são manipulados e alterados durante a execução de um algoritmo.

Logo, para que o resultado seja o esperado, é preciso considerar o tipo de dado que será armazenado e também definir o nome das variáveis.

Os valores armazenados nas variáveis são definidos em tempo de execução. Elas são associadas aos nomes; estes, definidos em tempo de desenvolvimento.

As variáveis são parte essencial do código desenvolvido em Javascript. Elas armazenam valores que são manipulados por nossos programas, permitindo que os algoritmos sejam implementados para dar a resposta esperada para o usuário.



O que são variáveis?

Tipos de declarações

Existem três tipos de declarações em JavaScript.

var

Declara uma variável, opcionalmente, inicializando-a com um valor.

let

Declara uma variável local de escopo do bloco, opcionalmente, inicializando-a com um valor.

const

Declare uma constante de escopo de bloco, apenas de leitura.

Um identificador JavaScript deve começar com uma letra, sublinhado (_), ou cifrão (\$); os seguintes caracteres também podem ser números (0-9). JavaScript diferencia maiúsculas de minúsculas, letras incluem caracteres de "A" devido a "Z" (maiúsculos) e caracteres de "a" a "z" (minúsculos).



O que são variáveis?

Declarando variáveis

Você pode declarar uma variável de três formas:

Com a palavra chave `var`. Por exemplo, `var x = 42`. Esta sintaxe pode ser usada para declarar tanto globais como globais globais.

Por simples adição de valor. Por exemplo, `x = 42`. Isso declara uma variável global. Essa declaração gera um aviso de declaração no JavaScript. Você não deve usar essa variação.

Com a palavra chave `let`. Por exemplo, `let y = 13`. Essa sintaxe pode ser usada para declarar uma variável local de escopo de bloco.



O que são variáveis?

Escopo de variáveis

Quando você declara uma variável para qualquer função, ela é chamada de variável global , porque está disponível para qualquer outro código no documento atual. Quando você declara uma variável dentro de uma função, é chamada de variável local , pois ela está disponível somente dentro dessa função.

```
if (true) {  
  var x = 5;  
}  
console.log(x); // 5
```

```
if (true) {  
  let y = 5;  
}  
console.log(y); // undefined, y não está definido
```



Tipos de Variáveis

Javascript dispõe de diversos tipos de variáveis, cada um adequado para armazenar um tipo de dado. Nesse ponto, é importante ressaltar a importância de escolher com atenção os dados guardados em uma variável.

O Javascript é uma linguagem fracamente tipada, (isto é, não força o programador a declarar qual o tipo de dado que a variável receberá em tempo de desenvolvimento.



Tipos de Variáveis

Números

Diferente de muitas linguagens de programação, o Javascript não diferencia números inteiros, em ponto flutuante, double precision, etc. Todos os números são tratados como ponto flutuante de 64 bits.

Para declarar um inteiro, por exemplo, usamos:

```
var inteiro = 5;
```

Declarando um número com casas decimais é feito:

```
var pi = 3.14;
```

Caso seja feita uma operação entre dois números inteiros que resulte em um número com casas decimais, o resultado será automaticamente interpretado corretamente:

```
var dividendo = 5;
```



Tipos de Variáveis

Strings

Strings são conjuntos de texto. São declaradas usando aspas simples ou aspas duplas:

```
var exemplo = 'Essa é uma string em Javascript';
```

```
var exemplo2 = "Outra string em Javascript";
```



Tipos de Variáveis

Booleans (booleanos)

Frequentemente, durante o desenvolvimento de código Javascript, haverá valores que, por definição, podem assumir apenas dois valores: verdadeiro ou falso.

Booleanos em Javascript descrevem exatamente esse comportamento. Uma variável do tipo boolean pode assumir apenas o valor true (verdadeiro) ou false (falso).

Variáveis deste tipo são usadas para executar operações lógicas, por exemplo, dentro de blocos if. Veja alguns exemplos:

```
var resultado1 = 5>4;
```

```
// O valor da variável resultado1 será true (verdadeiro), por que 5 é maior que 4.
```

```
var resultado2 = 4>5;
```

```
// O valor da variável resultado2 será false (falso), por que 4 não é maior que cinco.
```



Tipos de Variáveis

Arrays

Arrays são objetos de alto nível que descrevem listas de valores. Os arrays são capazes de armazenar diversos valores indexados por números inteiros.

Por exemplo, o array abaixo armazena uma lista de carros:

```
var carros = ["Gol", "Uno", "Agile"];
```

A indexação de arrays em Javascript começa com 0. É importante observar que, apesar de o tamanho do nosso array `carros` ser 3, o primeiro valor terá índice 0 e o último, índice 2.

Para acessar elementos de um array, é usada a sintaxe abaixo:

```
var gol = carros[0]; // O valor de gol será "Gol", o primeiro valor do array.
```



Tipos de Variáveis

Para mudar o valor de uma determinada posição de um array, usamos:

```
carros[2] = "Spin"; // O array agora será: ["Gol", "Uno", "Spin"];
```

Arrays também podem ser declarados usando a palavra-chave 'new':

```
var carros = new Array("Gol", "Uno", "Agile");
```

Um array pode receber entre seus elementos qualquer tipo de dado Javascript: strings, números e até mesmo objetos.

O poder dos arrays em Javascript está nos métodos e propriedades disponíveis para usarmos.



Tipos de Variáveis

Vejamos alguns exemplos de métodos e propriedades bastante úteis quando estamos desenvolvendo programas em Javascript.

Para saber o tamanho, isto é, a quantidade de elementos de um array, pode ser usada a propriedade 'length':

```
var tamanho = carros.length; // a variável tamanho vai guardar o valor 3
```

Caso precise ordenar os elementos de um array, pode ser usado o método 'sort':

```
carros.sort(); // O resultado será ["Agile", "Gol", "Uno"]  
[2, 1, 8, 5].sort() // O resultado será [1, 2, 5, 8]
```



Tipos de Variáveis

O método **push** é usado para incluir um elemento em um array. Ele sempre inclui o elemento no final do array:

```
carros.push("Fusca");  
// o array carros agora tem os seguintes valores ["Gol", "Uno", "Agile", "Fusca"]
```

Já o método **pop** apaga o último elemento do array:

```
var fusca = carros.pop();  
// O array carros agora virou ["Gol", "Uno", "Agile"]  
// E a variável fusca guarda a string (texto) "Fusca"
```

Note que o método **push** recebe como argumento o elemento que deve ser inserido no array.

O método **pop**, por outro lado, não recebe nenhum argumento, mas retorna o elemento excluído do array.



Tipos de Variáveis

Objetos

Nós já vimos que o Javascript permite criar variáveis para armazenar diversos tipos de dados. Objetos são tipos de dados especiais que podem armazenar vários valores ao mesmo tempo. Um objeto é uma abstração de um tipo complexo de dados.

Se quisermos manipular um tipo de dado que descreva um cliente, em Javascript, podemos usar um objeto para modelar esse cliente:

```
var cliente = {  
    nome:"João",  
    sobrenome:"Silva",  
    idade:50  
};
```

Para acessar as propriedades de objetos, usamos o operador de ponto:

```
var nomeCliente = cliente.nome; // nomeCliente será "João" ou var nomeCliente = cliente["nome"];
```




Tipos de Variáveis

Functions

Um recurso bastante interessante dos objetos em Javascript é a possibilidade de definir funções personalizadas. Ainda usando o exemplo do objeto cliente, podemos definir uma função que retorne o nome completo do cliente:

```
var cliente = {  
    nome: "João",  
    sobrenome: "Silva",  
    idade: 50,  
    nomeCompleto: function() {  
        return this.nome + " " + this.sobrenome;  
    }  
};
```



Tipos de Variáveis

A propriedade “nomeCompleto”, quando for chamada por meio do operador de ponto, fará a chamada da função que retornará a propriedade “nome”, concatenada com um espaço e a propriedade “sobrenome”:

```
var nomeFuncao = cliente.nomeCompleto();  
// a função retornará João Silva.
```

Observe que, por ser uma função, ao acessar a propriedade “nomeCompleto”, é preciso abrir e fechar parênteses, explicitando que o que se quer é que a função seja executada.



Constantes

Você pode criar uma constante apenas de leitura por meio da palavra-chave `const`. A sintaxe de um identificador de uma constante é semelhante ao identificador de uma variável: deve começar com uma letra, sublinhado ou cifrão e pode conter caractere alfabético, numérico ou sublinhado.

```
const PI = 3.14;
```

Uma constante não pode alterar seu valor por meio de uma atribuição ou ser declarada novamente enquanto o script está em execução. Deve ser inicializada com um valor.

As regras de escopo para as constantes são as mesmas para as variáveis let de escopo de bloco. Se a palavra-chave `const` for omitida, presume-se que o identificador represente uma variável.



Conversão de tipos de dados

JavaScript é uma linguagem dinamicamente tipada. Isso significa que você não precisa especificar o tipo de dado de uma variável quando declará-la, e tipos de dados são convertidos automaticamente conforme a necessidade durante a execução do script. Então, por exemplo, você pode definir uma variável da seguinte forma:

```
var answer = 42;
```

E depois, você pode atribuir uma string para a mesma variável, por exemplo:

```
answer = "Obrigado pelos peixes...";
```

Devido JavaScript ser dinamicamente tipado, essa declaração não gera uma mensagem de erro.



Conversão de tipos de dados

Em expressões envolvendo valores numérico e string com o operador +, JavaScript converte valores numérico para strings. Por exemplo, considere a seguinte declaração:

```
x = "A resposta é " + 42 // "A resposta é 42"  
y = 42 + " é a resposta" // "42 é a resposta"
```

Nas declarações envolvendo outros operadores, JavaScript não converte valores numérico para strings. Por exemplo:

```
"37" - 7 // 30  
"37" + 7 // "377"
```



Convertendo strings para números

No caso de um valor que representa um número está armazenado na memória como uma string, existem métodos para a conversão.

parseInt()

A função `parseInt()` analisa um argumento string e retorna um inteiro na base especificada.

parseFloat()

A função `parseFloat()` analisa um argumento string e retorna um número de ponto flutuante.

parseInt irá retornar apenas números inteiros, então seu uso é restrito para a casa dos decimais. Além disso, é uma boa prática ao usar **parseInt** incluir o parâmetro da base. O parâmetro da base é usado para especificar qual sistema numérico deve ser usado.